

Assignment_6.5

2303A510H5

Batch_30

Task_1: Use an AI tool to generate eligibility logic.

Prompt:

“Generate Python code to check voting eligibility based on age and citizenship.”

Code:

```
#voting eligibilty based on age and citizenship
age = int(input("Enter your age: "))
citizenship = input("Are you a citizen of the country? (yes/no): ").lower()
if age >= 18 and citizenship == "yes":
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

output:

```
Enter your age: 20
Are you a citizen of the country? (yes/no): yes
You are eligible to vote.
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

AI tools simplify logic generation by quickly producing accurate code for common eligibility checks. Automating this task ensures consistency and reduces

human error in applying rules like age and citizenship. It saves development time, allowing programmers to focus on more complex features instead of repetitive conditions.

Task_2:

Use an AI tool to process strings using loops.

Prompt:

“Generate Python code to count vowels and consonants in a string using a loop.”

Code:

```
#Generate Python code to count vowels and consonants in a string using a loop
string = input("Enter a string: ")
vowels = "aeiouAEIOU"
vowel_count = 0
consonant_count = 0
for char in string:
    if char.isalpha(): # Check if the character is an alphabet
        if char in vowels:
            vowel_count += 1
        else:
            consonant_count += 1
print(f"Number of vowels: {vowel_count}")
print(f"Number of consonants: {consonant_count}")
```

Output:

```
coding\lab6.5.py'
Enter a string: parimala
Number of vowels: 4
Number of consonants: 4
PS C:\Users\parim\Ai Assistant coding> |
```

Justification:

- Using a loop allows systematic traversal of each character in the string.
- Conditional checks (isalpha, membership in vowels) ensure accurate classification.
- This approach demonstrates how AI tools can automate repetitive logic creation efficiently.

Task_3: Use an AI tool to generate a complete program using classes, loops, and conditionals.

Prompt:

“Generate a Python program for a library management system using classes, loops, and conditional statements.”

Code:

```
class Library:
    def __init__(self):
        self.books = []

    def add_book(self, title):
        self.books.append(title)
        print(f"{title}" has been added to the library.")

    def display_books(self):
        if not self.books:
            print("No books in the library.")
        else:
            print("Books in the library:")
            for book in self.books:
                print(f"- {book}")

library = Library()
while True:
    print("\nLibrary Management System")
    print("1. Add a book")
    print("2. Display books")
    print("3. Exit")
    choice = input("Enter your choice (1/2/3): ")
    if choice == "1":
        title = input("Enter the book title: ")
        library.add_book(title)
    elif choice == "2":
        library.display_books()
    elif choice == "3":
        print("Exiting the program.")
        break
    else:
        print("Invalid choice. Please try again.")
```

Output:

```
Library Management System
1. Add a book
2. Display books
3. Exit
Enter your choice (1/2/3): 2
No books in the library.

Library Management System
1. Add a book
2. Display books
3. Exit
Enter your choice (1/2/3): 1
Enter the book title: star
"star" has been added to the library.

Library Management System
1. Add a book
2. Display books
3. Exit
Enter your choice (1/2/3): 2
Books in the library:
- star

Library Management System
1. Add a book
2. Display books
3. Exit
Enter your choice (1/2/3): 3
Exiting the program.
PS C:\Users\parim\Ai Assistant coding> █
```

Justification:

The program uses classes (Book and Library) to model real-world entities and their behaviours. Loops provide continuous interaction through the menu until the user exits. Conditionals handle user choices and book availability, ensuring logical flow in the system.

Task_4: Use an AI tool to generate an attendance management class.

Prompt: "Generate a Python class to mark and display student attendance using loops."

Code:

```
#Generate a Python class to mark and display student attendance using loops.
class Attendance:
    def __init__(self):
        self.attendance_record = {}

    def mark_attendance(self, student_name):
        self.attendance_record[student_name] = "Present"
        print(f"{student_name} has been marked as present.")

    def display_attendance(self):
        if not self.attendance_record:
            print("No attendance records available.")
        else:
            print("Attendance Records:")
            for student, status in self.attendance_record.items():
                print(f"{student}: {status}")

attendance = Attendance()
while True:
    print("\nStudent Attendance System")
    print("1. Mark Attendance")
    print("2. Display Attendance")
    print("3. Exit")
    choice = input("Enter your choice (1/2/3): ")
    if choice == "1":
        student_name = input("Enter the student's name: ")
        attendance.mark_attendance(student_name)
    elif choice == "2":
        attendance.display_attendance()
    elif choice == "3":
        print("Exiting the program.")
        break
    else:
        print("Invalid choice. Please try again.")
```

Output:

```
Student Attendance System
1. Mark Attendance
2. Display Attendance
3. Exit
Enter your choice (1/2/3): 1
Enter the student's name: parimala
parimala has been marked as present.

Student Attendance System
1. Mark Attendance
2. Display Attendance
3. Exit
Enter your choice (1/2/3): 2
Attendance Records:
parimala: Present

Student Attendance System
1. Mark Attendance
2. Display Attendance
3. Exit
Enter your choice (1/2/3): 3

Student Attendance System
1. Mark Attendance
2. Display Attendance
3. Exit
Enter your choice (1/2/3): 3
2. Display Attendance
3. Exit
Enter your choice (1/2/3): 3
Enter your choice (1/2/3): 3
Exiting the program.
PS C:\Users\parim\AI Assistant coding> █
```

Justification:

- The class encapsulates attendance logic, making it reusable and organized.
- Loops are used to iterate through student lists and mark attendance efficiently.
- Conditional handling ensures flexibility in marking and displaying records clearly.

Would you like me to extend this so it can calculate attendance percentage for each student as well? That would make it closer to a real-world system.

Task_5: Use an AI tool to complete a navigation menu.

Prompt: "Generate a Python program using loops and conditionals to simulate an ATM menu."

Code:

```
#Generate a Python program using loops and conditionals to simulate an ATM menu.
class ATM:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited: ${amount:.2f}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds.")
        else:
            self.balance -= amount
            print(f"Withdrew: ${amount:.2f}")

    def check_balance(self):
        print(f"Current balance: ${self.balance:.2f}")

atm = ATM()
while True:
    print("\nATM Menu")
    print("1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Exit")
    choice = input("Enter your choice (1/2/3/4): ")
    if choice == "1":
        amount = float(input("Enter amount to deposit: "))
        atm.deposit(amount)
    elif choice == "2":
        amount = float(input("Enter amount to withdraw: "))
        atm.withdraw(amount)
    elif choice == "3":
        atm.check_balance()
    elif choice == "4":
        print("Exiting the program.")
        break
    else:
        print("Invalid choice. Please try again.")
```

Output:

```
Python312\python.exe' 'c:\Users\parim\.vscode\extensions\ms-pyth
'--' 'c:\Users\parim\Ai Assistant coding\lab6.5.py'

ATM Menu
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1/2/3/4): 1
Enter amount to deposit: 2000
Deposited: $2000.00

ATM Menu
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1/2/3/4): 2
Enter amount to withdraw: 1000
Withdrew: $1000.00

ATM Menu
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1/2/3/4): 3
Current balance: $1000.00

ATM Menu
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1/2/3/4): 4
Exiting the program.
PS C:\Users\parim\Ai Assistant coding> █
```

Justification:

- ATM Menu Simulation – Using loops and conditionals for menu navigation demonstrates interactive program design, strengthening logic flow and user input handling in Python.